Continuing Research … → KVMs in small foot-print

RE: https://github.com/ahrink/debs

Recommended Small Footprint OS (apt-based)

1. Minideb (Best use case)

From: https://github.com/bitnami/minideb

Specification	Value
Size	~40-80 MB (uncompressed)
Base	Debian unstable/testing
apt	Full support
systemd	Optional (uses init system)
Best for	Scripted deployments, containers, minimal VMs

Download:

sh

```
# Latest stable
wget https://github.com/bitnami/minideb/releases/download/latest/minideb-buster-amd64.iso

# Or build your own (smallest)
docker pull bitnami/minideb:latest
```

Why it works: Built specifically for minimal Debian with apt. Perfect for AHR `apt-offline` bundle testing or AHR Downloader.

---

2. Slax (Debian Edition)

From: https://www.slax.org/

Specification	Value
Size	~300 MB
Base	Debian (stable)
apt	Supported
Live CD	Yes, with persistence
Best for	USB boot, portable environment

Download:

sh

```
wget https://www.slax.org/download.php -O slax-debian.iso
# Direct: https://download.slax.org/current/slax-64bit-11.6.0.iso
```

Why it works: Originally mentioned in AHR profiles (AHR Downloader), modular design, apt works after `apt-get update`.

---

3. MiniOS Live (SourceForge link)

From: https://sourceforge.net/projects/minios-live/

Specification	Value
Size	~200-400 MB
Base	Debian
apt	Supported
Modules	Yes (squashfs)
Best for	Custom live USB with persistence

Download:

sh

```
# Standard edition
wget https://sourceforge.net/projects/minios-live/files/Latest/MiniOS-Standard-amd64.iso/download

# Minimal (smallest)
wget https://sourceforge.net/projects/minios-live/files/Latest/MiniOS-Minimal-amd64.iso/download
```

Why it works: Modular, apt-based, can be stripped to ~150 MB.

Even Smaller Options (For Reference)

Debian Netinstall (Official)

sh

```
# Smallest official Debian (network install)
wget https://cdimage.debian.org/debian-cd/current/amd64/iso-cd/debian-12.5.0-amd64-netinst.iso
# Size: ~400-500 MB, but installs minimal system (~1 GB disk)
```

Ubuntu Minimal

sh

```
# Ubuntu minimal image
wget https://cdimage.ubuntu.com/ubuntu/releases/22.04/release/ubuntu-22.04-minimal-live-server-amd64.iso
# Size: ~1.2 GB (larger, but still minimal by Ubuntu standards)
```

DietPi (For ARM/Raspberry Pi)

sh

```
# DietPi - extremely small, but requires manual apt setup
wget https://dietpi.com/downloads/images/DietPi_RPi-ARMv8-Bookworm.7z
# Size: ~300 MB compressed, apt available after first boot
```

Recommended Testing Workflow

For fastest testing with bundles:

sh

```
#!/bin/sh
# download_minimal_os.sh - Get the smallest test environment

# Option 1: Minideb (recommended - 40MB)
echo "Downloading Minideb..."
wget -O minideb.iso
https://github.com/bitnami/minideb/releases/download/latest/minideb-buster-
amd64.iso

# Option 2: Slax (original target)
echo "Downloading Slax..."
wget -O slax.iso https://download.slax.org/current/slax-64bit-11.6.0.iso

# Option 3: MiniOS Minimal
echo "Downloading MiniOS Minimal..."
wget -O minios-minimal.iso
https://sourceforge.net/projects/minios-live/files/Latest/MiniOS-Minimal-
amd64.iso/download

echo ""
echo "Verifying downloads:"
ls -lh *.iso
```

Testing w/Bundles on Each OS

For Minideb (Fastest):

sh

```
# Boot Minideb ISO
# Login: root / no password (or bitnami/bitnami)

# Update apt sources (Minideb needs this but why buster and not jammy)
echo "deb http://deb.debian.org/debian buster main" > /etc/apt/sources.list
apt-get update

# Run w/install script
sh /mnt/install_bundles.sh
```

For Slax:

sh

```
# Boot Slax
# Enable apt (disabled by default)
apt-get update
apt-get install apt-offline

# Run your bundles
sh /path/to/install_bundles.sh
```

For MiniOS:

sh

```
# Boot MiniOS
# apt already enabled
apt-get install apt-offline
sh /path/to/install_bundles.sh
```

Comparison Table

OS	Size	apt-offline	Persistence	Best For
Minideb	~40 MB	Works	Needs setup	Script testing, CI/CD
Slax	~300 MB	Works	Built-in (USB)	Portable workstation
MiniOS	~200 MB	Works	Module-based	Custom live images
Debian Netinstall	~400 MB	Works	Full install	Production systems
DietPi	~300 MB	Manual	Full install	ARM/SBC devices

AHR Recommendation

For developing and testing your `apt-offline` bundles:

1. **Minideb** → Fastest boot, smallest download, pure apt
2. **Slax** → Your original target, real-world validation
3. **MiniOS** → Middle ground, good for USB persistence

Quick start with Minideb:

sh

```
# Download (40MB)
wget https://github.com/bitnami/minideb/releases/download/latest/minideb-buster-amd64.iso

# Boot in QEMU (if testing)
qemu-system-x86_64 -cdrom minideb-buster-amd64.iso -m 1024 -net user -net nic

# Inside VM:
echo "deb http://deb.debian.org/debian buster main" > /etc/apt/sources.list
apt-get update
apt-get install -y apt-offline
# Then run your install_bundles.sh
```

Important Notes

OS	Note
Minideb	No `sources.list` by default - must add
Slax	apt is present but modules are read-only

OS	Note
MiniOS	May need `apt-get install apt-offline` first
All	Test bundles in VM before production

Would you like me to create a **single test script** that:

1. Downloads all three minimal OS ISOs
2. Creates a QEMU/KVM test script for each
3. Automatically runs your bundle installation for validation?

well-structured approach:

- **POSIX-compliant** scripts (`#!/bin/sh`)
- **Simple `for` loops** (no fragile `IFS|` parsing)
- **Respected category order** (00_base → 05_editors)
- **Idempotent by design** (let `apt` handle duplicates)
- **Portable across any apt-based OS** (Slax, Minideb, MiniOS, Ubuntu Server)

Quick Reference Card for LAB Testing

sh

```
# Generate bundles (online machine)
sh v01_install.sh

# Install on target (offline, any apt OS)
sh v01_install.sh

# Verify key packages
dpkg -l | grep -E "tmux|vim|qemu-kvm|bridge-utils"
```

Test Order I'd Recommend

1. **Minideb** (fastest, 40MB) → validates apt-offline works
2. **Slax** (original target) → validates real-world use
3. **MiniOS** (persistence) → validates long-term deployment

Where offline machines become Debian-like fortresses.

```
# Generate each profile in order (respecting category structure)
echo ""
echo "=== Generating profile bundles ==="
echo "Portable media structure:"
```

```
echo "├── v01_online.sh            # Run on online machine"
echo "├── v01_install.sh           # Run on offline target"
echo "└── V01/"
echo "    ├── profiles/            # .sig files"
echo "    │   ├── update.sig"
echo "    │   ├── 00_base.sig"
echo "    │   ├── 01_networking.sig"
echo "    │   ├── 02_disk_utils.sig"
echo "    │   ├── 03_terminal.sig"
echo "    │   ├── 04_kvm.sig"
echo "    │   ├── 05_editors.sig"
echo "    │   └── apt_offline.sig" # tarball was generated w/AHR Downloader
echo "    └── tarballs/            # .tar.gz bundles"
echo "        ├── 00_base.tar.gz"
echo "        ├── 01_networking.tar.gz"
echo "        ├── apt_offline.tar.gz"
echo "        └── …"
```

San Anton [20260407 via AHR “KVMs in small foot-print”]

PS: the final repo after results indicating “*working as expected*” is often forgot. Just copy /var/cache/apt/archives/*.deb to a /tmp/result-dir/ – make a portable tarball and save it on the external drive (good memories implied) — REM: the ISO should allow for DATA partition on same disk although the OS is read only. So we may need to edit some of these small foot-print OS ISOs. Primary Partition vs DATA on ext4 FS.

See downloader as a beginning research results:
https://sananton.substack.com/p/bootstrapping-the-offline-world

Scripts are here: https://github.com/ahrink/debs